

Practice Project: Book Browser Web Application

Project Overview

In this project, you will build an interactive **Book Browser** web application using **HTML, CSS, modern JavaScript, and DOM manipulation**.

The Book Browser is a front-end web app that allows users to browse, search, filter, and view information about books in a clean and interactive interface. The purpose of the project is to demonstrate your ability to build a dynamic web application that updates the page in response to user actions without reloading.

This is **not** a static website. Your application must include real JavaScript logic and meaningful DOM updates.

What is the Book Browser?

The **Book Browser** is a website where users can explore a collection of books.

The app should allow users to:

- view a list of books
- search for a book by title
- filter books by category or genre
- click a book to see more details
- interact with the page through buttons, forms, filters, and dynamic content

You will create the book data yourself in JavaScript. You may store the data as an **array of objects** inside your JavaScript file.

Each book should contain information such as:

- title
 - author
 - category / genre
 - year
 - image / cover
 - short description
 - rating or availability status (optional but recommended)
-

Project Goal

Your goal is to create a **clean, interactive, and professional-looking Book Browser** that clearly demonstrates your skills in:

- modern JavaScript
 - DOM selection and manipulation
 - event handling
 - dynamic rendering
 - working with arrays and objects
 - functions and logic
 - form and input handling
 - code organization and best practices
-

Required User Interface (UI)

Your Book Browser application should include the following main interface sections.

1. Header

The page should include a header with:

- the application title
Example: **Book Browser**
- a short subtitle or description
- a clean navigation or toolbar area if needed

Purpose:

This section introduces the app and makes it look like a real product.

2. Search and Filter Section

You must include a section where the user can interact with the app.

This section must include:

- a **search input** to search by book title
- at least **one filter control**, such as:
 - dropdown for category/genre

- buttons for categories
- rating filter
- year filter

Purpose:

This section allows users to narrow down the visible books and interact with the dataset.

3. Book List / Book Cards Section

This is the main display area of the app.

Requirements:

- display multiple books dynamically using JavaScript
- each book should appear as a card, box, or list item
- each displayed item should show at least:
 - book cover image
 - title
 - author
 - category

Purpose:

This section shows your ability to render content dynamically into the DOM.

4. Book Details Section

When the user clicks a book, the page should show more information.

This section should display:

- title
- author
- category
- year
- full or longer description
- cover image
- any additional details you want to include

This can appear:

- in a separate section on the same page
- in a popup/modal-like section
- or in a dedicated details area below the book list

Purpose:

This demonstrates event handling and dynamic content updates.

5. Footer

Include a footer with simple project information such as:

- student name
 - course name
 - project title
-

Required Functionality

Your Book Browser must support the following features.

A. Display Books Dynamically

- Use JavaScript to render the books into the DOM
- Do not hard-code all book cards directly in HTML
- Book data should come from JavaScript objects/arrays

B. Search Functionality

- Users must be able to type into a search box
- The displayed book list should update based on the search input
- The search should at least work by **book title**

C. Filter Functionality

- Users must be able to filter books by at least one category
- When a filter is selected, only matching books should appear

D. Book Details Display

- Clicking a book should show additional details dynamically
- This should be handled using JavaScript and DOM updates

E. Interactive Behavior

The app must react to user actions such as:

- typing
 - clicking
 - selecting filters
 - clearing or changing search/filter values
-

Technical Requirements

1. Use Modern JavaScript

Your project must demonstrate:

- let and const
- arrays and objects
- conditions
- loops or array methods
- functions
- event listeners
- template literals where appropriate
- clear and modular logic

2. DOM Manipulation

You must use JavaScript to:

- select elements
- update content
- create or render book cards dynamically
- update book details dynamically
- change visible results based on input

3. CSS Styling

Your Book Browser should be styled using CSS and should look clean and modern.

Your design should include:

- readable typography
- spacing and alignment
- styled cards or list items
- styled search/filter controls
- organized layout
- consistent colors and appearance

The app should look like a polished student portfolio piece, not just a raw demo.

4. File Organization

Your files must be organized professionally.

Suggested structure:

```
book-browser/  
├── index.html  
├── css/  
│   └── styles.css  
├── js/  
│   └── app.js  
└── images/
```

You may organize it slightly differently, but it must be clear and logical.

AI Usage Policy

Students are **allowed to use AI tools** to help with this project.

AI may be used for:

- brainstorming UI ideas
- generating starter code

- creating sample book objects
- helping build rendering functions
- debugging search or filter logic
- improving CSS styling
- explaining JavaScript behavior
- refactoring code

However:

Students are responsible for:

- understanding the code they submit
- testing all features
- validating AI-generated results
- making sure the final project meets all requirements

AI should be used as a **helper and learning assistant**, not as a replacement for student understanding.

Git and GitHub

You are encouraged to use Git and GitHub to track your progress.

- Create a GitHub repository for the project
- Upload all source files
- Commit your work throughout development
- Recommended: ~10–15 meaningful commits

Examples of meaningful commit history:

- Create initial project structure
- Add base HTML layout
- Add book data array
- Render book cards dynamically
- Add search functionality
- Add category filtering
- Add book details display
- Improve card styling
- Refactor rendering functions
- Add footer and cleanup
- Improve responsive layout
- Final fixes and comments

Important:

- Do **not** upload the entire project in one commit
 - Commit messages should be meaningful and reflect real development progress
-

Code Quality

Your project must follow good coding practice. You should aim to:

- use meaningful variable and function names
- write clear and organized code
- separate logic into functions where appropriate
- avoid unnecessary repetition
- include enough comments to explain major sections and important logic
- follow clean code and best practices

Comments

You do not need to comment every line, but you **must include enough comments** to explain:

- the book data section
 - rendering logic
 - search/filter logic
 - event handling
 - details display logic
 - any tricky or non-obvious code
-

Expected Final Output

When the application runs, the user should see:

- a clean Book Browser page
- a search area and filter controls
- a dynamically displayed list of books
- clickable book entries
- a details area that changes based on the selected book
- smooth interaction without page reloads
- an organized and readable layout

The final result should feel like a small but complete front-end application.

Recommended Output

You are encouraged to:

- Build a fully working Book Browser application
- Organize your project files clearly
- Use GitHub to track your progress with meaningful commits
- Ensure all features (search, filter, details) are working

(Optional)

- Save screenshots or a short demo of your application

These steps will help you build a **strong portfolio-ready project**.

Student Checklist

Before wrapping up your project, try to ensure:

- a working Book Browser app
- JavaScript-generated book list
- search functionality
- filter functionality
- clickable book details
- DOM updates based on user interaction
- clean CSS styling
- organized files
- enough comments
- GitHub repository
- at least 15 commits

How to Evaluate Your Work

Use the following points to assess your progress:

- Does your Book Browser work correctly?
- Are search, filter, and book details features functioning as expected?
- Is your content rendered dynamically using JavaScript (not hard-coded)?
- Does the app respond properly to user interactions?
- Is your code clear, organized, and easy to understand?
- Does the application look clean and usable?
- Have you used GitHub to track your progress with meaningful commits?
- Does your project feel complete enough to include in a portfolio?